



Reporte 2. Vista general de los lenguajes de programación.

Angel Rugerio Jiménez #201720

Axel García Arellano #201251

Lenguajes de Computación – Grupo “A”

Prof. Paulo Daniel Vazquez Mora

Septiembre 24, 2026

Un lenguaje de programación es una notación formal con la que se describen cómputos, de modo que un mismo texto sea comprensible para quien lo escribe y, a la vez, ejecutable por una computadora. Esa doble audiencia explica casi toda su historia: cada generación de lenguajes intenta acercar la notación al problema que el programador piensa y alejarla de los detalles del hardware que la ejecuta. A ese movimiento se le llama **abstracción**: la evolución de los paradigmas y lenguajes de programación "marca un uso creciente de la abstracción" (Garrido, 2019, p. 1). Es el hilo conductor de este informe.

La relación con la materia es directa. Todo lenguaje de programación tiene una **sintaxis** definida por una gramática formal —normalmente libre de contexto, tipo 2 en la jerarquía de Chomsky— y sus componentes léxicos (identificadores, números, palabras reservadas) se reconocen con expresiones regulares y autómatas finitos, tipo 3. La teoría de lenguajes formales no es un tema aparte de los lenguajes de programación: es la herramienta con la que se especifican.

El documento se organiza en cinco apartados: (a) historia, (b) proceso de abstracción, (c) abstracciones comunes, (d) tipos de lenguajes y (e) paradigmas, seguidos de las conclusiones y referencias.

a. Historia de los lenguajes de programación

a.a Lenguajes pioneros

- **Código máquina y ensamblador (finales de los 40 – 50).** Las primeras computadoras de programa almacenado se programaban con secuencias binarias de instrucciones. El ensamblador sustituyó los códigos numéricos por mnemónicos (MOV, ADD, JMP) y las direcciones por etiquetas simbólicas, traducidos uno a uno por un *ensamblador*. Fue el primer paso de abstracción, aunque cada programa seguía dependiendo de un procesador específico.
- **Fortran (1957).** Desarrollado en IBM por el equipo de John Backus, fue el primer lenguaje de alto nivel de uso amplio. Su mayor logro fue demostrar que un compilador podía generar código comparable en eficiencia al escrito a mano, argumento que convenció a los programadores escépticos de su tiempo. Introdujo expresiones aritméticas en notación algebraica, arreglos y el ciclo DO.
- **Lisp (1958–1960).** John McCarthy lo diseñó para la investigación en inteligencia artificial. Aportó la lista como estructura universal, la recursión como mecanismo principal de control, las funciones como valores, la recolección automática de basura y la idea de que código y datos comparten la misma representación (McCarthy, 1960).
- **ALGOL 60 (1960).** Surgió de un comité internacional europeo-estadounidense que comenzó a trabajar en 1958 (ALGOL 58 fue el prototipo). No fue un éxito comercial, pero fue el primer lenguaje cuya sintaxis se definió formalmente, mediante la **notación**

BNF (Backus–Naur Form), una gramática libre de contexto. Introdujo bloques anidados, alcance léxico y procedimientos recursivos en un lenguaje imperativo, y es el ancestro de Pascal, C y la mayoría de los lenguajes imperativos modernos (MSMK University College, 2025).

- **COBOL (1960).** Orientado al procesamiento de datos de negocios, con sintaxis cercana al inglés y registros como estructura central. El comité CODASYL publicó su especificación en 1959, y los primeros compiladores estuvieron disponibles al año siguiente.

a.b Principales hitos en la evolución

- **Programación estructurada (finales de los 60 – 70).** La carta de Dijkstra (1968) contra la instrucción goto impulsó programar solo con secuencia, selección e iteración. Pascal (1970) y C (1972) consolidaron el estilo; C además se volvió el lenguaje de sistemas por excelencia con Unix.
- **Abstracción de datos y módulos (70).** Parnas (1972) formuló el principio de **ocultamiento de información**: un módulo expone una interfaz y esconde las decisiones de diseño que pueden cambiar. Modula-2 y Ada lo llevaron al lenguaje.
- **Orientación a objetos (1967 – 80).** Simula 67 introdujo clases y objetos para simulación; Smalltalk (Xerox PARC, década de 1970) llevó la idea al extremo —todo es un objeto que se comunica mediante mensajes— (Kay, 1993). C++ (1985) y después Java (1995) la llevaron a la industria.
- **Programación lógica (1972).** Prolog, creado por Colmerauer y Roussel en Marsella, permitió programar declarando hechos y reglas en lógica de primer orden y dejando la búsqueda de soluciones al sistema (Colmerauer & Roussel, 1993).
- **Programación funcional (70 – 90).** La conferencia Turing de Backus (1978) criticó el "cuello de botella de von Neumann" de los lenguajes imperativos y propuso un estilo funcional. ML (1973) aportó inferencia de tipos; Haskell (1990) consolidó la evaluación perezosa y la pureza, dos ideas que facilitan construir programas a partir de piezas pequeñas y combinables.
- **Lenguajes de script y la web (90).** Python (1991), JavaScript (1995), PHP y Ruby priorizaron la productividad: tipado dinámico, interpretación y bibliotecas extensas.

a.c Tendencias actuales

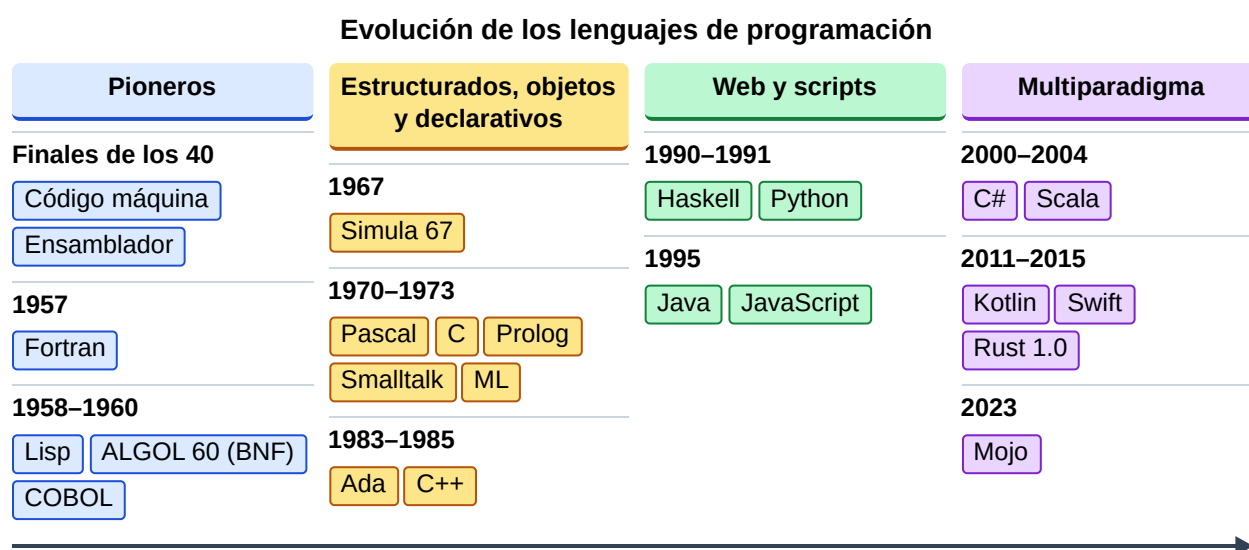
- **Multiparadigma como norma.** Lenguajes de uso general recientes (Scala, Kotlin, Swift, Rust) y la evolución de los clásicos (Java 8 con lambdas, C++11, C# con LINQ) integran en un mismo lenguaje objetos, funciones de orden superior e inmutabilidad.
- **Seguridad de memoria sin recolector de basura.** Rust usa un sistema de *ownership* verificado en compilación para eliminar clases enteras de errores de memoria que en C y C++ se detectaban en ejecución.

- **Lenguajes para inteligencia artificial y cómputo científico.** Python domina por su ecosistema (NumPy, PyTorch), aunque las operaciones costosas se ejecutan en C/C++/CUDA; Julia y Mojo —presentado por la empresa Modular en 2023— buscan unir la productividad de Python con el rendimiento de un lenguaje compilado. En finanzas cuantitativas, por ejemplo, Mojo se propone para evitar reescribir en C++ los modelos investigados en Python (Han, 2026).
- **Lenguajes de dominio específico (DSL).** SQL, HTML/CSS, expresiones regulares, Terraform o los lenguajes de consulta de datos resuelven un problema acotado con una notación diseñada específicamente para él. La tendencia se extiende a la IA: NineToothed es un DSL construido sobre Triton que deja escribir núcleos de GPU con semántica secuencial y genera el código paralelo (Huang et al., s.f.), y Anka restringe deliberadamente su sintaxis para que los modelos de lenguaje generen código de flujos de datos con menos errores (Al Mazrouei, 2025).

a.d Línea del tiempo

La evolución se lee en tres vistas complementarias: una línea del tiempo por etapas, un árbol de influencias entre lenguajes y una tabla cronológica con los datos de cada uno.

Línea del tiempo por etapas



Árbol de influencias (color = paradigma principal; línea punteada = influencia indirecta)

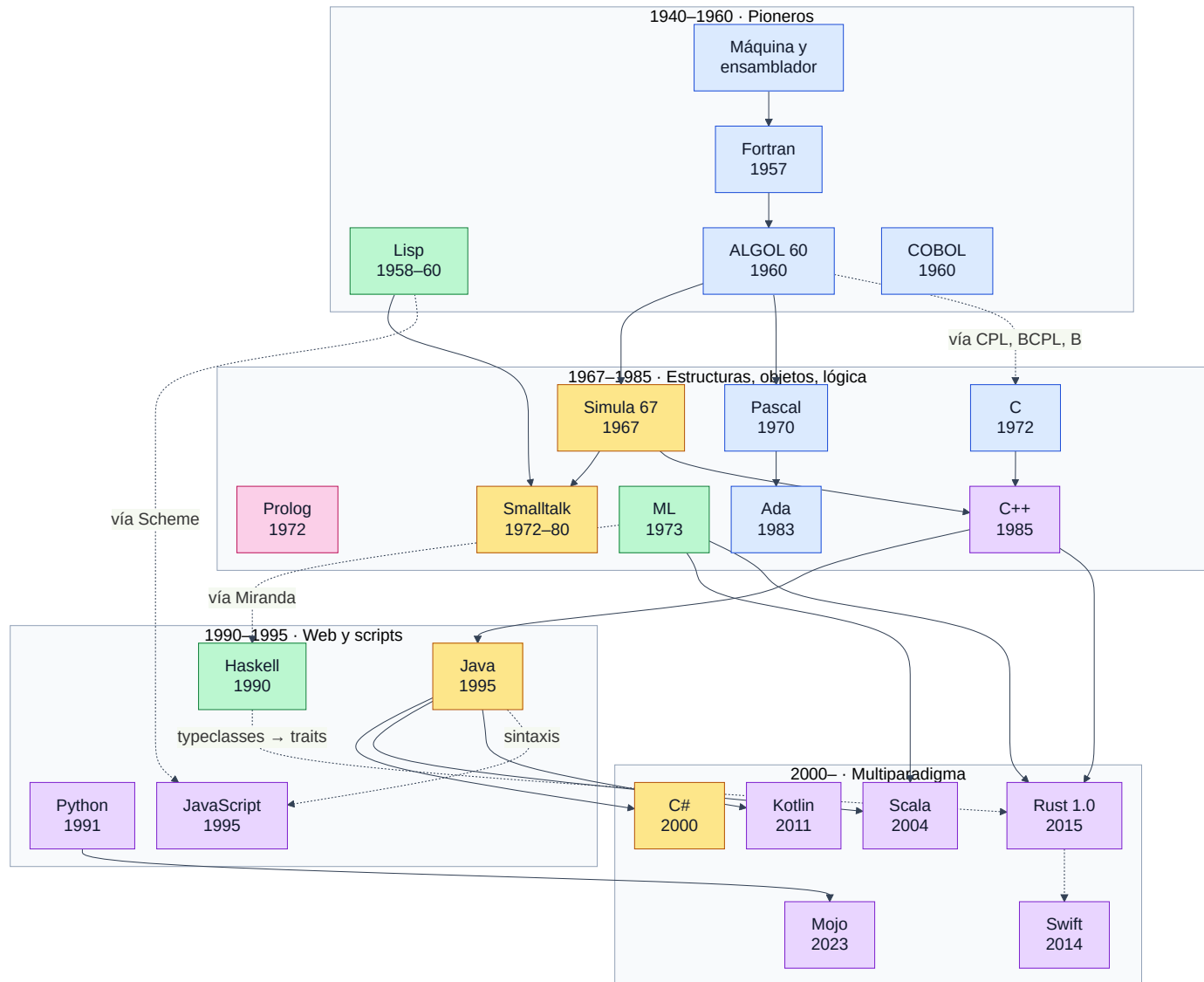


Tabla cronológica

Año	Lenguaje	Creadores / institución	Aporte principal	Paradigma
Finales de los 40	Código máquina	Primeras computadoras de programa almacenado	Programa almacenado en memoria	Imperativo
~1949–1950	Ensamblador	—	Mnemónicos y etiquetas simbólicas	Imperativo
1957	Fortran	John Backus, IBM	Primer compilador de alto nivel eficiente	Imperativo
1958–1960	Lisp	John McCarthy, MIT	Funciones como datos, recursión, recolector de basura	Funcional
1960	ALGOL 60	Comité internacional	Sintaxis formal con BNF, bloques, alcance léxico	Imperativo estructurado
1960	COBOL	Comité CODASYL	Procesamiento de datos de negocios, registros	Imperativo
1967	Simula 67	O.-J. Dahl y K. Nygaard, Noruega	Clases, objetos, herencia	Orientado a objetos
1970	Pascal	Niklaus Wirth, ETH Zúrich	Programación estructurada para la enseñanza	Imperativo estructurado
1972	C	Dennis Ritchie, Bell Labs	Lenguaje de sistemas portable (Unix)	Imperativo estructurado
1972	Prolog	A. Colmerauer y P. Roussel, Marsella	Hechos, reglas y unificación	Lógico
1972–1980	Smalltalk	Alan Kay et al., Xerox PARC	Todo es objeto, paso de mensajes	Orientado a objetos
1973	ML	Robin Milner, Edimburgo	Inferencia de tipos	Funcional
1983	Ada	Departamento de Defensa de EE. UU.	Módulos (paquetes), tipado fuerte	Imperativo estructurado
1985	C++	Bjarne Stroustrup, Bell Labs	Objetos y abstracción sobre C	Multiparadigma
1990	Haskell	Comité académico	Pureza y evaluación perezosa	Funcional
1991	Python	Guido van Rossum, CWI	Productividad, tipado dinámico	Multiparadigma
1995	Java	James Gosling, Sun Microsystems	Máquina virtual, portabilidad	Orientado a objetos
1995	JavaScript	Brendan Eich, Netscape	Programación en el navegador	Multiparadigma

Año	Lenguaje	Creadores / institución	Aporte principal	Paradigma
2000 (1.0 en 2002)	C#	Anders Hejlsberg, Microsoft	Plataforma .NET, LINQ (2007)	Orientado a objetos
2004	Scala	Martin Odersky, EPFL	Objetos y funcional sobre la JVM	Multiparadigma
2011 (1.0 en 2016)	Kotlin	JetBrains	Java más conciso y seguro ante null	Multiparadigma
2014	Swift	Apple	Sucesor de Objective-C	Multiparadigma
2015	Rust (1.0)	Mozilla	Seguridad de memoria en compilación	Multiparadigma
2023	Mojo	Modular	Sintaxis de Python con rendimiento de sistemas	Multiparadigma

Fechas y aportes de los lenguajes hasta 1995 según las fuentes históricas citadas en a.a y a.b; para los lenguajes posteriores a 2000 se indica el año de presentación pública y, entre paréntesis, el de la versión 1.0 cuando difiere.

b. Proceso de abstracción

b.a Qué significa abstraer

En computación, **abstraer** es quedarse con los aspectos de un problema que importan para un fin concreto y dejar de lado, a propósito, los que no. Garrido (2019) lo formula como "un proceso mental que consiste en realzar los detalles relevantes [...] mientras se ignoran los detalles irrelevantes", lo que reduce la información que hay que manejar en un momento dado y permite "tratar cosas diferentes como si fueran la misma" (p. 1). Un lenguaje de programación es, en sí mismo, una abstracción de la máquina: ofrece un conjunto de conceptos (variables, funciones, objetos) que el traductor convierte en instrucciones concretas.

Se suelen distinguir dos grandes clases de abstracción:

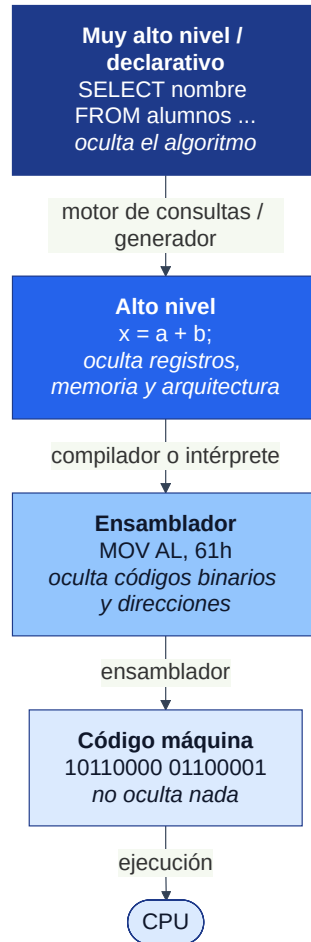
- **Abstracción de procesos:** encapsular una secuencia de acciones bajo un nombre (subprogramas, funciones, métodos). El que llama sabe *qué* hace, no *cómo*. Es el primer tipo de abstracción que aparece históricamente (Garrido, 2019, p. 3).
- **Abstracción de datos:** definir un tipo junto con las operaciones permitidas sobre él, ocultando su representación (tipos abstractos de datos, clases, módulos). Un tipo de dato abstracto es "una colección de valores junto con unas operaciones sobre ellos, definidos mediante una especificación que es independiente de cualquier implementación" (Garrido, 2019, p. 6).

También pueden clasificarse por su alcance, desde un elemento aislado hasta una unidad completa de programa: **básicas** (una sola variable, un solo paso), **estructuradas** (arreglos, ciclos, registros) o **de unidad** (módulos, paquetes, componentes que agrupan código y datos).

b.b Niveles de abstracción

Cada nivel se traduce al anterior; subir un nivel reduce la cantidad de decisiones que toma el programador y aumenta las que toma el traductor.

Nivel	Ejemplo	Qué oculta	Traductor
Máquina	10110000 01100001	Nada	— (lo ejecuta la CPU)
Ensamblador	MOV AL, 61h	Códigos binarios y direcciones numéricas	Ensamblador
Alto nivel	x = a + b;	Registros, memoria, arquitectura concreta	Compilador o intérprete
Muy alto nivel / declarativo	SELECT nombre FROM alumnos WHERE semestre = 5;	El algoritmo: se dice <i>qué</i> se quiere, no <i>cómo</i> obtenerlo	Motor de consultas, generador



Más arriba = más abstracción: menos decisiones del programador, más decisiones del traductor. La traducción es conceptual: un motor de consultas no genera código C, pero sí ejecuta el plan de consulta con código compilado de nivel inferior.

El mismo cálculo —sumar los elementos de una lista— en tres niveles:

```

suma.asm (x86-64, ensamblador)
    xor    rax, rax          ; acumulador = 0
    xor    rcx, rcx          ; índice = 0
ciclo:
    cmp    rcx, rdx          ; rdx = longitud
    jge    fin
    add    rax, [rsi+rcx*8]   ; rsi = dirección del arreglo
    inc    rcx
    jmp    ciclo
fin:
  
```

suma.c (alto nivel, imperativo)

```
long suma(const long *xs, int n) {
    long total = 0;
    for (int i = 0; i < n; i++)
        total += xs[i];
    return total;
}
```

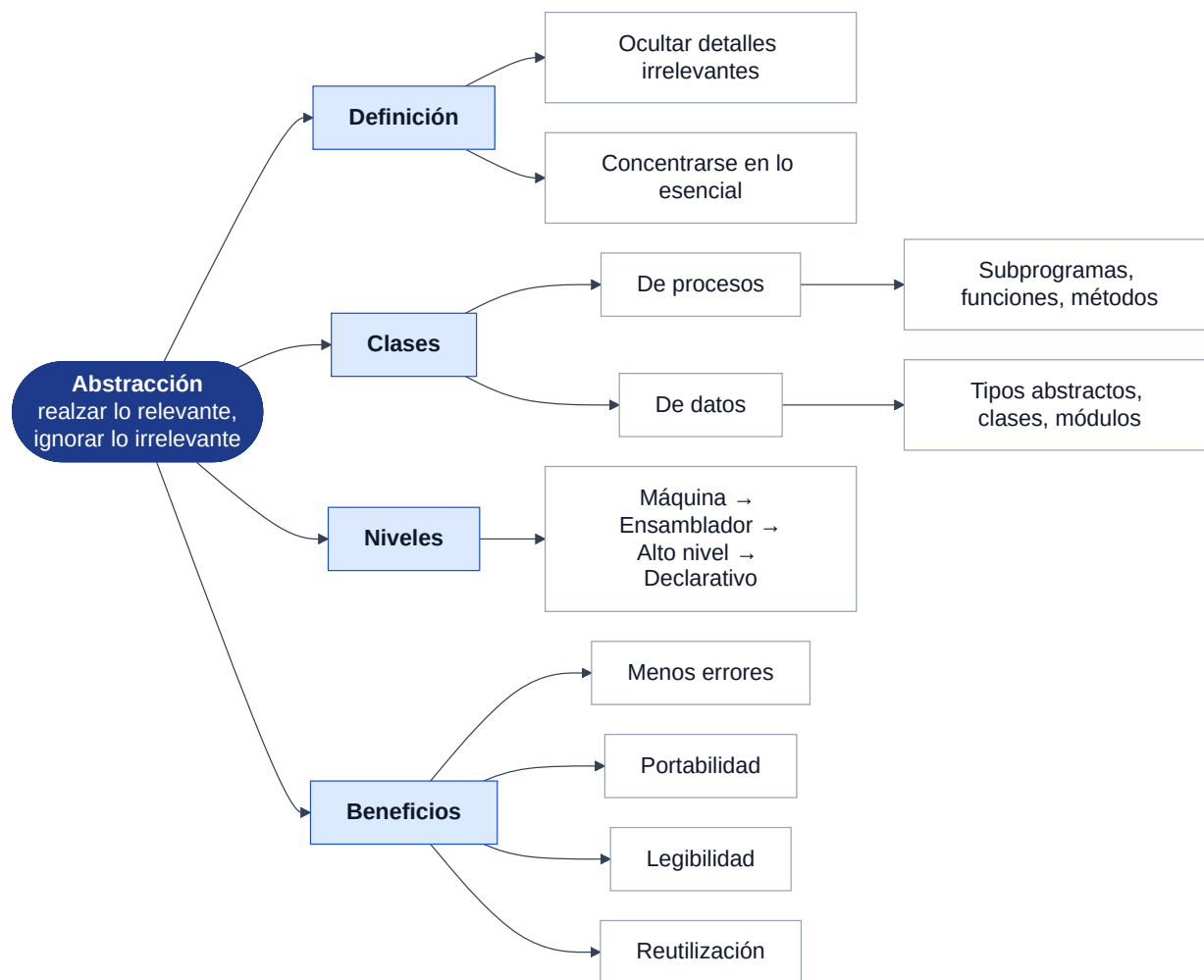
suma.py (muy alto nivel)

```
total = sum(xs)
```

b.c Cómo la abstracción simplifica la programación

- **Menos errores:** el compilador asigna registros y calcula direcciones; el programador ya no puede equivocarse en eso.
- **Portabilidad:** el mismo programa en C o Java corre en arquitecturas distintas; el ensamblador no.
- **Legibilidad y mantenimiento:** `sum(xs)` expresa la intención; el ciclo en ensamblador obliga a reconstruirla.
- **Reutilización:** una función, una clase o un módulo se escriben una vez y se usan muchas.
- **Ejemplo de la materia:** en la Actividad 6 la función de transición $\delta: Q \times \Sigma \rightarrow Q$ se representó como un diccionario anidado de Python (`tabla[estado][simbolo]`). La abstracción de datos del lenguaje permitió escribir el autómata casi igual que su definición formal, sin gestionar memoria ni estructuras de bajo nivel.

b.d Cuadro sinóptico del proceso de abstracción



c. Abstracciones más comunes en los lenguajes

Abstracción	Qué representa	Ejemplo
Variable	Nombre asociado a una celda de memoria con un valor, un tipo, un alcance y un tiempo de vida	<code>int edad = 20;</code>
Tipo de dato	Conjunto de valores más las operaciones válidas sobre ellos	<code>int, float, bool, str, arreglos, registros</code>
Estructura de control	Orden en que se ejecutan las instrucciones	<code>if, while, for, match</code>
Función / procedimiento	Secuencia de instrucciones con nombre y parámetros	<code>def area(r): return 3.1416 * r ** 2</code>
Objeto	Entidad con estado (atributos) y comportamiento (métodos)	<code>cuenta.depositar(100)</code>

Abstracción	Qué representa	Ejemplo
Clase	Plantilla que define la estructura y el comportamiento de un conjunto de objetos	<code>class Cuenta: ...</code>
Módulo	Unidad que agrupa definiciones y expone una interfaz, ocultando la implementación	<code>import math</code> , paquetes de Java
Concurrencia	Ejecución de varias tareas cuyos tiempos se traslapan	hilos, <code>async/await</code> , canales, actores

- **Variables.** Abstraen la dirección de memoria. Sus atributos son nombre, dirección, valor, tipo, tiempo de vida y alcance; el momento en que cada atributo queda fijo se conoce como *binding* (ligadura), y puede darse durante la compilación o durante la ejecución.
- **Tipos de datos.** Abstraen la representación binaria. Un sistema de tipos puede ser **estático** (se verifica en compilación: C, Java, Rust) o **dinámico** (en ejecución: Python, JavaScript), y **fuerte** o **débil** según cuánto permite mezclar tipos sin conversión explícita.
- **Estructuras de control.** Abstraen los saltos del hardware. Con secuencia, selección e iteración se puede expresar cualquier algoritmo, lo que respalda la programación estructurada (Dijkstra, 1968).
- **Funciones y procedimientos.** Abstracción de procesos: el que llama conoce la firma, no el cuerpo. Mecanismos de paso de parámetros: por valor, por referencia, por nombre.
- **Objetos y clases.** Unen datos y operaciones. La clase es la definición; el objeto, la instancia. Encapsulamiento, herencia y polimorfismo son sus tres pilares.
- **Módulos.** Abstracción de unidad. Siguen el principio de ocultamiento de información de Parnas (1972): cada módulo esconde una decisión de diseño susceptible de cambiar.
- **Concurrencia.** Abstrae la ejecución paralela o intercalada. Modelos: memoria compartida con hilos y candados, paso de mensajes (CSP; Hoare, 1978), base de los canales de Go,, actores (Erlang) y `async/await` para E/S.

d. Tipos de lenguajes de programación

Criterio	Bajo nivel	Alto nivel	Dominio específico (DSL)
Cercanía	A la máquina	Al programador	Al problema
Ejemplos	Código máquina, ensamblador	C, Java, Python, Haskell	SQL, HTML, regex, LaTeX, Verilog
Portabilidad	Nula (depende del procesador)	Alta	Depende del motor que lo interpreta
Control del hardware	Total	Parcial (C) a nulo (Python)	Nulo
Productividad	Baja	Alta	Muy alta dentro de su dominio

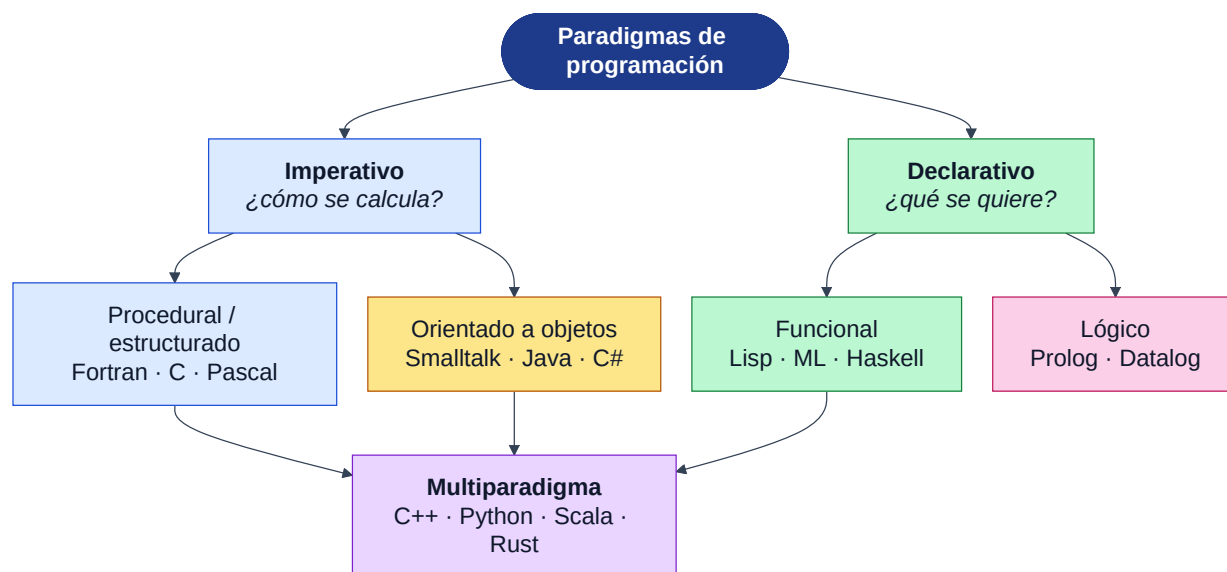
Criterio	Bajo nivel	Alto nivel	Dominio específico (DSL)
Alcance	General	General (Turing completo)	Restringido, a veces no Turing completo
Traducción	Ensamblador	Compilador, intérprete o híbrido (bytecode + VM)	Motor propio o biblioteca anfitriona

- **Bajo nivel.** Las instrucciones corresponden casi uno a uno con las de la CPU. Se usan en *firmware*, controladores, sistemas embebidos, rutinas críticas de rendimiento y análisis de *malware*. Relación con la materia: el curso de Lenguajes de Computación explica por qué los mnemónicos de un ensamblador se reconocen con un autómata finito.
- **Alto nivel.** Más cercanos al lenguaje humano que al hardware: usan instrucciones más abstractas y menos dependientes de la infraestructura física de la computadora (Lenovo, s.f.). Son independientes de la arquitectura y de propósito general. Se subdividen según su forma de ejecución: **compilados** (C, Rust, Go), **interpretados** (Python, Ruby) e **híbridos** (Java y C# compilan a *bytecode* para una máquina virtual).
- **De dominio específico.** Tienen un poder expresivo deliberadamente reducido, pensado para un área de problemas concreta. Pueden ser **externos** (con sintaxis y analizador propios, como SQL o las expresiones regulares) o **internos** (construidos dentro de un lenguaje anfitrión, como las consultas de SQLAlchemy en Python). Diseñar un DSL implica elegir un punto dentro de un *espacio de diseño*: una colección de lenguajes relacionados descrita por dimensiones a menudo ortogonales (Zong et al., 2021). Las expresiones regulares son el DSL más cercano a esta materia: describen exactamente los lenguajes regulares que reconoce un autómata finito.

e. Paradigmas de programación

Un **paradigma** es un modelo o estilo de programación: un conjunto de conceptos con los que se piensa y se estructura una solución (Floyd, 1979). En la práctica, un mismo lenguaje suele admitir varios paradigmas, y lo que distingue a cada uno son los conceptos que pone a

disposición del programador (estado, funciones de orden superior, concurrencia, etc.), más que la sintaxis.



Clasificación clásica en dos familias. Existen taxonomías más finas que distinguen los paradigmas según los conceptos que incluye cada uno; esta figura es una simplificación didáctica.

e.a Imperativo

Describe **cómo** se llega al resultado mediante instrucciones que modifican el estado del programa (asignación). Es el reflejo directo de la arquitectura de von Neumann: variables = celdas de memoria, asignación = escritura, ciclos = saltos (Backus, 1978). Su variante **estructurada** prohíbe el goto y organiza el código en bloques y procedimientos. Lenguajes: Fortran, C, Pascal.

```
factorial.c
int factorial(int n) {
    int r = 1;
    for (int i = 2; i <= n; i++) r *= i;
    return r;
}
```

e.b Orientado a objetos

Organiza el programa como un conjunto de **objetos** que encapsulan estado y comportamiento y colaboran enviándose mensajes (Kay, 1993). Pilares: encapsulamiento, herencia y polimorfismo, más la abstracción mediante clases e interfaces. Lenguajes: Smalltalk, Java, C#, C++.

```
cuenta.py
class Cuenta:
    def __init__(self, saldo=0):
        self._saldo = saldo          # estado encapsulado

    def depositar(self, monto):
        self._saldo += monto
```

e.c Funcional

El programa es una composición de **funciones matemáticas puras**: el resultado depende solo de los argumentos y no hay efectos secundarios ni asignación destructiva. Las funciones son valores de primera clase, la repetición se expresa con recursión o funciones de orden superior (map, filter, fold). Una de sus ventajas principales es la **modularidad**: las funciones de orden superior y la evaluación perezosa permiten combinar piezas pequeñas de maneras difíciles de lograr en el estilo imperativo. Lenguajes: Lisp, ML, Haskell, Erlang.

```
factorial.hs
factorial :: Integer -> Integer
factorial 0 = 1
factorial n = n * factorial (n - 1)
```

e.d Lógico

El programa es un conjunto de **hechos** y **reglas** en lógica de primer orden; la ejecución consiste en hacerle una consulta al sistema, que busca una demostración mediante unificación y retroceso (*backtracking*). El programador declara *qué* es verdad, no *cómo* buscarlo (Colmerauer & Roussel, 1993). Lenguaje principal: Prolog; también Datalog.

```
familia.pl
padre(juan, ana).
padre(ana, luis).
abuelo(X, Z) :- padre(X, Y), padre(Y, Z).
% ?- abuelo(juan, Quien).    -> Quien = luis.
```

e.e Multiparadigma

Un lenguaje multiparadigma ofrece conceptos de varios paradigmas para que el programador elija el adecuado a cada parte del problema. Python admite estilo imperativo, objetos y funciones de orden superior; Scala y Kotlin mezclan objetos con funcional; C++ combina imperativo, objetos y programación genérica; Rust une imperativo, funcional y un sistema de tipos con *traits*.

```

multiparadigma.py
# el mismo cálculo en dos estilos dentro de un solo lenguaje
total = 0
for x in xs:                                # imperativo
    total += x * x

total = sum(map(lambda x: x * x, xs))      # funcional

```

e.f Tabla comparativa de paradigmas

Criterio	Imperativo	Orientado a objetos	Funcional	Lógico
Pregunta central	¿Cómo se calcula?	¿Qué objetos colaboran?	¿Qué función transforma la entrada?	¿Qué es verdad?
Unidad básica	Instrucción	Objeto / clase	Función pura	Hecho / regla
Estado	Mutable, explícito	Mutable, encapsulado	Inmutable	No hay estado explícito
Control	Secuencia, ciclos	Mensajes entre objetos	Recursión, composición	Unificación y <i>backtracking</i>
Fundamento	Máquina de von Neumann	Simulación, modelado	Cálculo lambda	Lógica de primer orden
Estilo	Procedural	Procedural + encapsulado	Declarativo	Declarativo
Lenguajes	Fortran, C, Pascal	Smalltalk, Java, C#	Lisp, Haskell, ML	Prolog, Datalog
Aplicación típica	Sistemas, cómputo numérico	Aplicaciones grandes, GUI, empresas	Transformación de datos, concurrencia	Sistemas expertos, bases de conocimiento

e.g Paradigmas emergentes (opcional)

- **Reactivo:** el programa reacciona a flujos de eventos que cambian en el tiempo (RxJS, frameworks de interfaces).

- **Orientado a datos / array programming:** operaciones sobre arreglos completos en vez de elementos (NumPy, APL, Julia), base del cómputo de IA.
- **Programación probabilística:** los modelos estadísticos se escriben como programas y la inferencia la resuelve el sistema (Stan, Pyro). La idea ya se aplica a la generación de código con modelos de lenguaje: un solo programa probabilístico puede representar de forma compacta muchos programas deterministas candidatos (Garg et al., 2026).
- **Programación diferenciable:** los programas pueden derivarse automáticamente para entrenarse con gradiente (JAX, PyTorch).

f. Conclusiones

La historia de los lenguajes de programación es la historia de la abstracción: del código binario al ensamblador, de ahí a los lenguajes de alto nivel y, finalmente, a notaciones declarativas y de dominio específico en las que se describe el problema y no el procedimiento. Cada paso transfirió decisiones del programador al traductor, a cambio de más legibilidad, portabilidad y menos errores.

Los paradigmas no compiten entre sí; son formas complementarias de organizar un programa. La tendencia actual lo confirma: los lenguajes modernos son multiparadigma y el programador competente es el que sabe qué concepto usar en cada parte del problema.

Por último, la teoría de lenguajes formales que estudia esta materia es la base técnica de todo lo anterior: gramáticas libres de contexto para la sintaxis, expresiones regulares y autómatas finitos para el análisis léxico. Entender un lenguaje de programación a fondo exige entender los lenguajes formales con los que se define.

Referencias

- Al Mazrouei, S. K. S. (2025). *Anka: A domain-specific language for reliable LLM code generation* [Preprint]. University of Wisconsin-Madison.
- Backus, J. (1978). Can programming be liberated from the von Neumann style? A functional style and its algebra of programs. *Communications of the ACM*, 21(8), 613–641. <https://doi.org/10.1145/359576.359579>
- Barberousse, P. (2008). Fundamentos teóricos del pensamiento complejo de Edgar Morin. *Revista Educare*, 12(2), 95–113.
- Chakray. (2025, 24 de febrero). *Lenguajes de programación: tipos y características*. <https://chakray.com/es/lenguajes-programacion-tipos-caracteristicas/>
- Colmerauer, A., & Roussel, P. (1993). The birth of Prolog. *ACM SIGPLAN Notices*, 28(3), 37–52. <https://doi.org/10.1145/155360.155362>
- Dijkstra, E. W. (1968). Letters to the editor: Go to statement considered harmful. *Communications of the ACM*, 11(3), 147–148. <https://doi.org/10.1145/362929.362947>
- Floyd, R. W. (1979). The paradigms of programming. *Communications of the ACM*, 22(8), 455–460. <https://doi.org/10.1145/359138.359140>
- Garg, P., Geh, R. L., Israel, D., Millstein, T., & Van den Broeck, G. (2026). *Probabilistic programs of thought* [Preprint].
- Garrido, A. (2019). *Abstracción en programación* [Informe]. Departamento de Ciencias de la Computación e Inteligencia Artificial, Universidad de Granada.
- Han, H. (2026). *Mojo: A promising tool for scalable financial AI efficiency* [Preprint]. arXiv. <https://arxiv.org/abs/2606.16059>
- Hoare, C. A. R. (1978). Communicating sequential processes. *Communications of the ACM*, 21(8), 666–677. <https://doi.org/10.1145/359576.359585>
- Huang, J., Li, Z., Li, Y., & Wang, H. (s.f.). *NineToothed: A Triton-based high-level domain-specific language for machine learning* [Preprint].
- Kay, A. C. (1993). The early history of Smalltalk. *ACM SIGPLAN Notices*, 28(3), 69–95. <https://doi.org/10.1145/155360.155364>
- Learning.com Team. (2025, 11 de febrero). *Understanding abstraction: Everyday examples and the role of abstraction in computational thinking*. Learning.com. <https://www.learning.com/blog/examples-of-abstraction-in-everyday-life/>
- Lenovo. (s.f.). *Lenguaje de alto nivel: todo lo que necesitas saber*. <https://www.lenovo.com/mx/es/glosario/lenguaje-de-alto-nivel/>
- McCarthy, J. (1960). Recursive functions of symbolic expressions and their computation by machine, Part I. *Communications of the ACM*, 3(4), 184–195. <https://doi.org/10.1145/367177.367199>

- MSMK University College. (2025, 14 de febrero). *Algol*. <https://msmk.university/algol/>
- Parnas, D. L. (1972). On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12), 1053–1058. <https://doi.org/10.1145/361598.361623>
- Zong, J., Pollock, J., Wootton, D., & Satyanarayan, A. (2021). Design spaces of domain-specific languages: Comparing and contrasting approaches in PL and HCI. En *PLATEAU: 12th Annual Workshop at the Intersection of PL and HCI* (pp. 1–11).